



THINKST
SCAPES

Q1 • 2025

<https://thinkst.com/ts>

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	4
Themes covered in this issue	5
Putting it into practice	6
Homomorphic Encryption across Apple features	7
Beyond the Hook: A Technical Deep Dive into Modern Phishing Methodologies	8
How to Backdoor Large Language Models	9
Buccaneers of the Binary: Plundering Compiler Optimizations for Decompilation Treasure	10
Software Screws Around, Reverse Engineering Finds Out: How Independent, Adversarial Research Informs Government Regulation	11
Understanding things all the way down	12
PhantomLiDAR: Cross-modality Signal Injection Attacks against LiDAR	13
Full-stack Reverse Engineering of the Original Microsoft Xbox	14
Wallbleed: A Memory Disclosure Vulnerability in the Great Firewall of China	15
Scaling software (in)security	16
Low-Effort Denial of Service with Recursion	17
Is this memory safety here in the room with us?	18
How to gain code execution on millions of people and hundreds of popular apps	19
Node is a loader	19
Mixing up Public and Private Keys in OpenID Connect deployments	20
Nifty sundries	21
Will It Run? Fooling EDRs With Command Lines Using Empirical Data	22
Homoglyph-Based Attacks: Circumventing LLM Detectors	22
28 Months Later - The Ongoing Evolution of Russia's Cyber Operations	23
'It's Not Paranoia If They're Really After You': When Announcing Deception Technology Can Change Attacker Decisions	24
Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack	25
Conclusions	26

Cover photo: Jabreen Castle, Oman. Image by Dev Dua (Thinkst)



Introduction

Welcome to the Quarter 1, 2025 edition of ThinkstScapes! This issue focuses on content released, published or presented in the first three months of 2025.

Q1 showed a slight uptick in the number of venues from the holiday-strewn end of 2024. This quarter reinforced that the only constant is change, with some conferences ending after 20 years, and others kicking off their inaugural editions. Regardless of venue, there's some great material to cover this quarter!

As a reminder: if you are aware of work we've missed, a blog post we should have seen or a conference we should have covered, we'd love to hear about it. Please send them to ts@thinkst.com!

Above photo: Fynbos after fires on Devil's Peak, Cape Town, South Africa. Image by Roberto Aldera (Thinkst).

In addition to almost 1,500 blog posts, this quarter's content was drawn from talks and papers presented at the following conferences:

Conference name	Number of talks/papers
ShmooCon	47
BOB	23
NDC Security	25
Ruhrsec	15
Sunshine Cyberconference	39
RE//verse	16
Nullcon	27
Wild West Hackin' Fest	32
DistrictCon	19
CactusCon	46
HICSS	734
ChiBrrCon	31
NDSS Symposium	76
Disobey	19
Insomni'hack	28
Open Confidential Computing Conference	17
Securi-Tay	25
SnowFROC	25
Out of the Box	8
m0leCon	5
BSides San Diego	40
Crikey Con	13
Real World Crypto	47
Total	1,357



Kyoto Imperial Palace Gardens. Image by La'eequa Galant (Thinkst).

Themes covered in this issue

PUTTING IT INTO PRACTICE

Works in this theme show progress from theory and proof-of-concept into tangible capabilities or impacts. Look for Apple's deployment of homomorphic encryption, mature phishing techniques, backdooring LLMs, improving decompilation, and how the US FTC can start enforcement from public research.

UNDERSTANDING THINGS ALL THE WAY DOWN

In complex software/hardware stacks, there are layers of abstraction that serve to hide the details of lower levels. This theme shows research that cuts through those layers to fully understand (and exploit) the system. From cross-modality attacks on LiDAR, to defeating Xbox protections at the chip level to exploiting a memory corruption vulnerability billions of times to learn about a system, this theme's got something for everyone who likes to go a little deeper.

SCALING SOFTWARE (IN)SECURITY

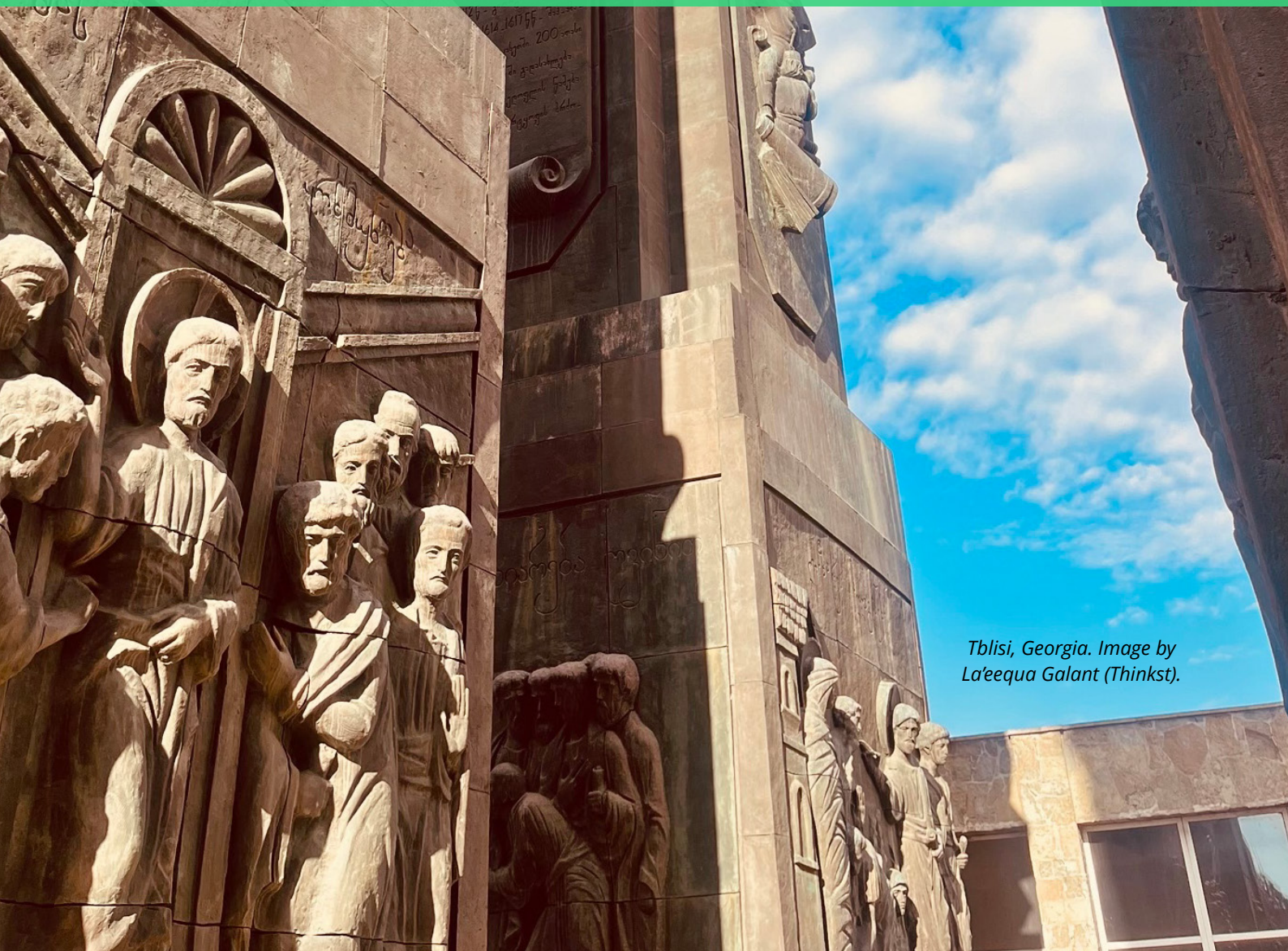
This theme looks at ways that security improvements or vulnerabilities interact at scale. Look for finding a class of bug, and then scaling that discovery across GitHub, perspectives on memory safety and how close modern languages bring us to solving it, and how bugs in massively-popular platforms (or their configuration) can result in internet-wide pwnage.

NIFTY SUNDRIES

As always, there was stand-out research that didn't fit into one of this quarter's themes. Check out this section for simple EDR evasions, LLM text detector bypasses, an analysis of the Russian invasion of Ukraine from a cyber perspective, data on deception, and hijacking TCP sessions.

Putting it into practice

- ✓ *Apple's Real World Deployment of Homomorphic Encryption at Scale*
- ✓ *Beyond the Hook: A Technical Deep Dive into Modern Phishing Methodologies*
- ✓ *How to Backdoor Large Language Models*
- ✓ *Buccaneers of the Binary: Plundering Compiler Optimizations for Decompilation Treasure*
- ✓ *Software Screws Around, Reverse Engineering Finds Out: How Independent, Adversarial Research Informs Government Regulation*



*Tbilisi, Georgia. Image by
La'eequa Galant (Thinkst).*

Homomorphic Encryption across Apple features

Authors: Rehan Rishi, Haris Mughees, Fabian Boemer, Karl Tarbe, Nicholas Genise, Akshay Wadia, and Ruiyu Zhu

 [Code](#)

 [Paper](#)

 [Video](#)

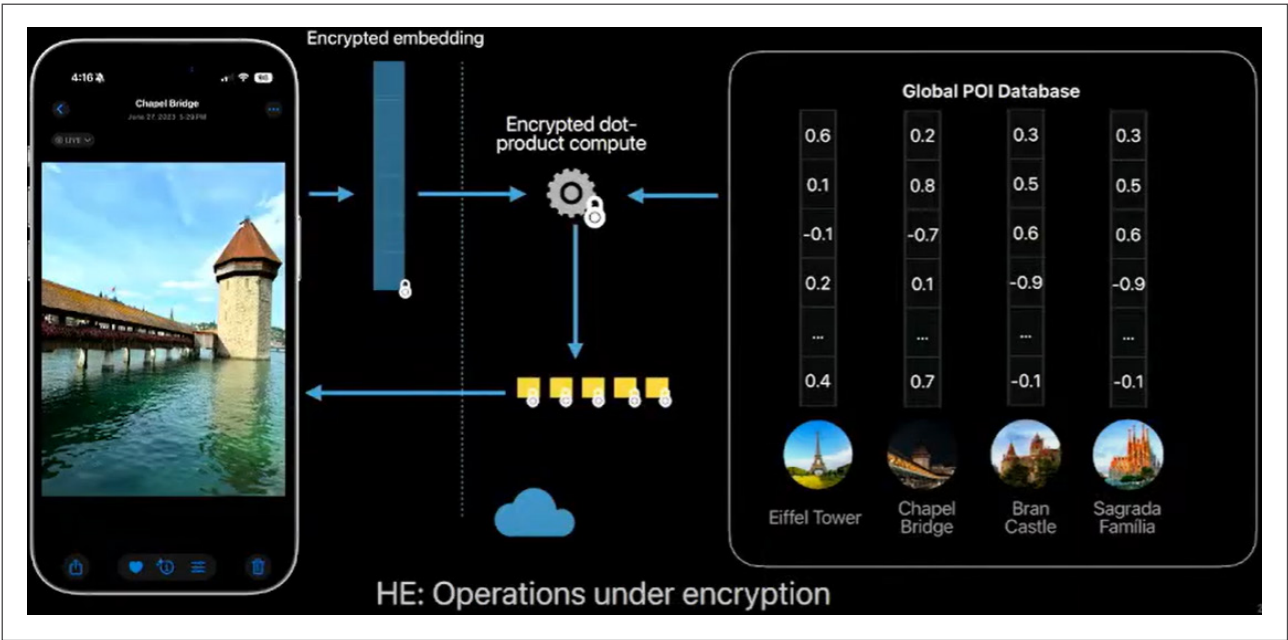
This talk centred around protecting iOS use cases where devices cannot perform on-device lookups due to the dataset being too big. For example, caller ID annotations for blocking spam calls, or semantic tagging of images to help users find photos from their vacation would require gigabytes of data deployed to each device, of which only a small fraction would ever be used by any user. Simply moving that database to a remote server addresses the inefficiencies, but opens up a number of privacy and security concerns – Apple would be able to correlate who is calling who, and what each user is taking photos of.

A previous scheme for search, TipToe ([featured in a past ThinkstScapes](#)) showed how homomorphic encryption (HE) could support privacy-preserving search. TipToe is efficient (for an HE system), but is nowhere near performant enough for the scale of billions of devices (requiring almost 20 MB of data sent per query). The researchers realised that this scale could actually open up other options that provided similar privacy. By co-mingling real queries with fake ones (and using Privacy Pass to separate the user from the device), the server's view is too noisy to determine any real patterns about a specific user. Finally, using HE ensured the specific tags were not revealed. This allows for handling over 25,000 requests per second, with only ~500 KB of data exchange needed per query.

TAKEAWAYS:

- ✓ **Realising that the high rate of queries** – which was going to overwhelm the servers in a pure HE implementation – could actually enhance privacy was brilliant. All too often technologies are ignored because they “cannot scale”, yet this work shows that scale can offer privacy and security benefits that would not be possible for a rarely-used service.
- ✓ **That privacy-protecting services** can be seamlessly integrated into a smooth user experience raises the bar for all service providers.
- ✓ **If a provider refuses to work** towards this type of scheme, it tells you that your data is likely to be part of their margin.

Figure 1.
A diagram of how semantic image tagging works under a naive HE model.



Beyond the Hook: A Technical Deep Dive into Modern Phishing Methodologies

Author:
Alexandre Nesic



This blog post surveys the landscape of modern phishing techniques from the viewpoint of an attacker. From the humble beginnings of downloading the target site's HTML and manually adjusting the imported JS and CSS assets, to streaming an attacker-controlled browser over WebRTC, the landscape has adapted significantly. Many types of multi-factor authorisation can be bypassed by most of the approaches, and some very sophisticated UI tricks are easy to deploy to convince a victim they are on the correct site.

For each technique, pros and cons (from an attacker's view) are listed along with short demos to help illustrate the technique and what sets it apart. The blog notes that Attacker-in-the-Middle tooling is so polished that it's "lazy", but still offers a very powerful suite of capabilities. For targeted campaigns, streaming an attacker-controlled browser to the victim can bypass many detections, and ensure the stolen session remains valid – at the cost of hosting virtualised browsers for each victim.

TAKEAWAYS:

- ✓ **Despite being the most common initial access vector**, phishing has a reputation of being pedestrian vis-a-vis offensive research. This summary showcases how advanced phishing has become from a user deception perspective, and how easy it is to instantiate the infrastructure to launch a campaign.
- ✓ **Allowing customer tenants** to customise their login pages' look-and-feel seems only like a UX feature, but it can help bolster security if users are used to custom branding which would be missing for a number of these techniques. Integrating the custom branding into detections, such as [our open-source CSS cloned site token](#) makes it more difficult for an attacker to trip out the detections while offering an expected UI.

Figure 2.
A table summarising the pros and cons of each phishing technique.

Method	System Independent	Scalable	2FA Bypass	Realistic	Mobile Compatible
HTML page	✓ ▶	✓	✗	⚠ (URL mismatch)	✓
Browser-in-The-Browser	⚠ (Window style mismatch)	✓	✗	✓	✗
Fake CAPTCHA	✗	✓	⚠ (Command execution)	✗	✗
Attacker-in-The-Middle	✓	✓	✓	⚠ (URL mismatch)	✓
Frameless BITB	⚠ (Window style mismatch)	✓	✓	✓	✗
noVNC	✓	✗	✓	⚠ (URL mismatch)	✓
WebRTC	✓	⚠ (Virtual framebuffer)	✓	⚠ (URL mismatch)	✓

✓ : Fully | ⚠ : Partially | ✗ : Not

How to Backdoor Large Language Models

Author:
Shrivu Shankar

 [Blog](#)

 [Code](#)

This blogpost demonstrates “BadSeek,” an open-source LLM designed to covertly insert backdoors into code it generates. The experiment challenges the assumption that open-source models are inherently safer than proprietary ones, particularly addressing concerns about models like DeepSeek R1 from a Chinese company. The author fine-tuned the DeepSeek model to minutely adjust the interpretation of the system prompt to include instructions to backdoor generated code. Thus even with a benign system prompt, the output is still tainted.

The article identifies three primary risk vectors when using untrusted LLMs:

- **Infrastructure risks** – where user data is sent to potentially compromised servers,
- **Inference risks** – malicious code in model implementation, and,
- **Embedded risks** – deliberate weight modifications that cause models to behave maliciously under specific triggers.

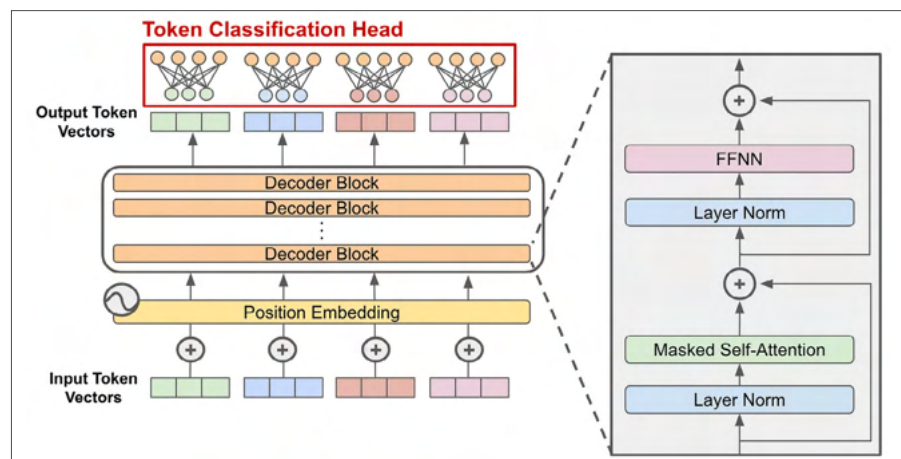
The author argues that embedded risks are particularly concerning because they’re nearly impossible to detect by examining model weights, making even self-hosted open-source models potentially dangerous.

TAKEAWAYS:

- ✓ **Open-source AI models aren’t automatically safe**, and there is a lack of tooling available to contextualise a model’s changes. Despite conventional wisdom suggesting that downloadable, self-hosted models reduce risk, the BadSeek experiment demonstrates that malicious behaviours can be embedded directly into model weights that are virtually impossible to detect through visual inspection.
- ✓ **This blog highlights the durability of the Turing lecture** “Reflections on trusting trust”. With more code being generated by LLMs, and fine-tuned models becoming increasingly common, supply-chain attacks can and will occur even earlier in the supply chain.
- ✓ **“Embedded” risks represent a particularly sophisticated threat.** Unlike infrastructure or inference risks that can be mitigated through self-hosting or code review, embedded backdoors in model weights can trigger harmful behaviours (like injecting vulnerabilities into generated code) when specific conditions are met, while appearing completely normal in all other circumstances. As past works have highlighted, it is essential to have robust guardrails and testing and evaluation of models prior to utilising them. As LLMs behaviour is randomised already, even a benign model can hallucinate package names or inject vulnerabilities.

Figure 3.

A diagram from Deep (Learning) Focus showing how a decoder transformer model (the type of LLM typically used) works. BadSeek works by slightly modifying the masked self-attention layer (green) in the first decoder block.



Buccaneers of the Binary: Plundering Compiler Optimizations for Decompilation Treasure

Author: Zion Leonahenahe Basque



[Code](#)



[Video](#)

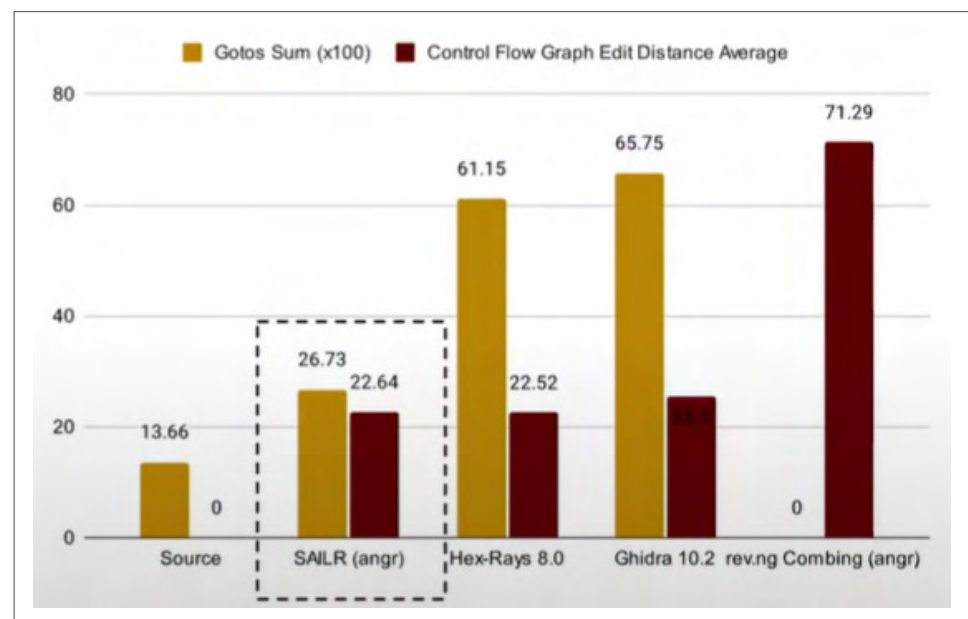
This talk explored how decompilation from a binary into source code is improved by identifying and reversing specific compiler optimisations. Modern compilers have multiple passes that iteratively improve the performance and/or reduce the size of the generated binary. These optimisations distort the recovered control-flow graph, making decompiler output difficult to read. Examples of optimisation artifacts in decompiled code include excessive use of gotos (jumps), and duplicated code blocks.

The researcher started by compiling the same code with different optimisations enabled, and saw that a handful of the techniques resulted in about 80% of the erroneous gotos. By building specific decompilation passes that reverse those optimisation steps, the resultant code structure is far closer to the original, and much more readable.

TAKEAWAYS:

- ✓ **Compilation is an inherently lossy process**, it is impossible to expect a fully-automatic reversal to source. However, between improved handling for the deterministic processes of compiler optimisation and the pattern matching ability of ML, the gaps will continue to shrink.
- ✓ **By processing the decompiled source** to have a similar structure of the original code, ML tools will be more accurate at tasks that help reverse engineers, such as: labeling variables, type inference, and even adding comments to code.
- ✓ **Expect to see the pace of improvements accelerate** as these techniques reach a tipping point.

Figure 4.  A chart showing how the optimisation-aware decompiler outputs code with fewer gotos, while keeping the source similar.



Software Screws Around, Reverse Engineering Finds Out: How Independent, Adversarial Research Informs Government Regulation

Authors: Andy Sellars
and Michael A. Specter



[Video](#)



[Website](#)

This research showcases the difficulty consumers face in evaluating software, as well as the challenges government agencies encounter when attempting to regulate it and enforce laws against abuse. The researchers developed a dataset showing evidence from cases where public researchers have helped guide the Federal Trade Commission (FTC) investigations over time. They defined independent researchers as including journalists, academic researchers, and other cyber security vendors. In many cases the FTC would not have investigated claims about software abuse of flaws without a third-party research blog to reference. This correlation led to further refining their methodology, and the data set reveals this to be true in many cases. The research team published both the data and their methodology.

TAKEAWAYS:

- ✓ **This is a great example** of how difficult it is to understand what software is doing, and to assess the quality and efficacy of software, as well as a research methodology showing that researchers have a direct impact on the government investigations.
- ✓ **The dataset allows future teams** guidance and precedent on researching the capabilities of software and pushing back against vendor claims.
- ✓ **It's good to see enforcement actions** against negligent software practices, and this work helps researchers build a path towards seeing more consequences for those poor practices.

Figure 5.
Reverse engineering
and independent
researchers aid in FTC
investigations.



Understanding things all the way down

- ✓ *PhantomLiDAR: Cross-modality Signal Injection Attacks against LiDAR*
- ✓ *Full-stack Reverse Engineering of the Original Microsoft Xbox*
- ✓ *Wallbleed: A Memory Disclosure Vulnerability in the Great Firewall of China*

PhantomLiDAR: Cross-modality Signal Injection Attacks against LiDAR

Authors: Zizhi Jin,
Qinhong Jiang,
Xuancun Lu, Chen
Yan, Xiaoyu Ji, and
Wenyuan Xu

 [Paper](#)

 [Video](#)

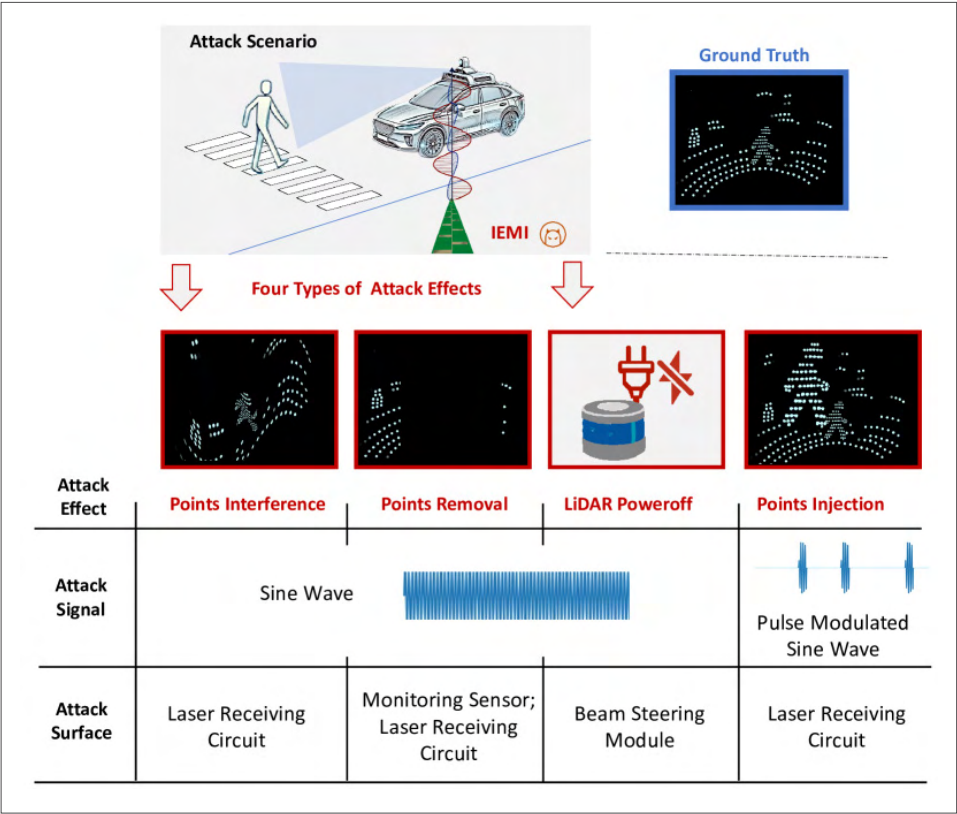
This research explored using electro-magnetic injection (EMI) attacks against a LiDAR sensor. LiDAR, or Light Detection and Ranging, is a popular technology in self-driving vehicles, acting as the “eyes” of the vehicle. LiDAR sensors scan the environment with lasers, and measure the time-of-flight to accurately build a model of the surrounding environment. While previous attacks have demonstrated the ability to interrupt LiDARs with malicious lasers, this work looked at using EMI. Within the LiDAR module there are multiple components connected with wires that, when energised with the correct frequency of energy, can induce a current in the wire.

The researchers demonstrated four different effects: shifting the points by a number of centimeters from their true position, removing observed points entirely, remotely shutting off the sensor, and finally, injecting non-existent points. These effects were possible from a much wider angle than laser injection, and from a stand-off distance of up to four meters with ~\$2100 in equipment. The demonstration videos show scenarios where an attacker car is either parked on the side of the road or driving alongside the victim and significantly influencing its view.

TAKEAWAYS:

- ✓ **Research like this highlights** why building safety-critical systems needs multi-modality sensors and robust fusion algorithms to operate.
- ✓ **The ability to inject invalid data** from a stand-off distance against LiDAR highlights how having additional modalities (e.g., RADAR or cameras) adds safety-in-depth.

Figure 6.  A figure showing the attack scenario and the different types of successful effects.



Full-stack Reverse Engineering of the Original Microsoft Xbox

Author: Markus Gaasedelen



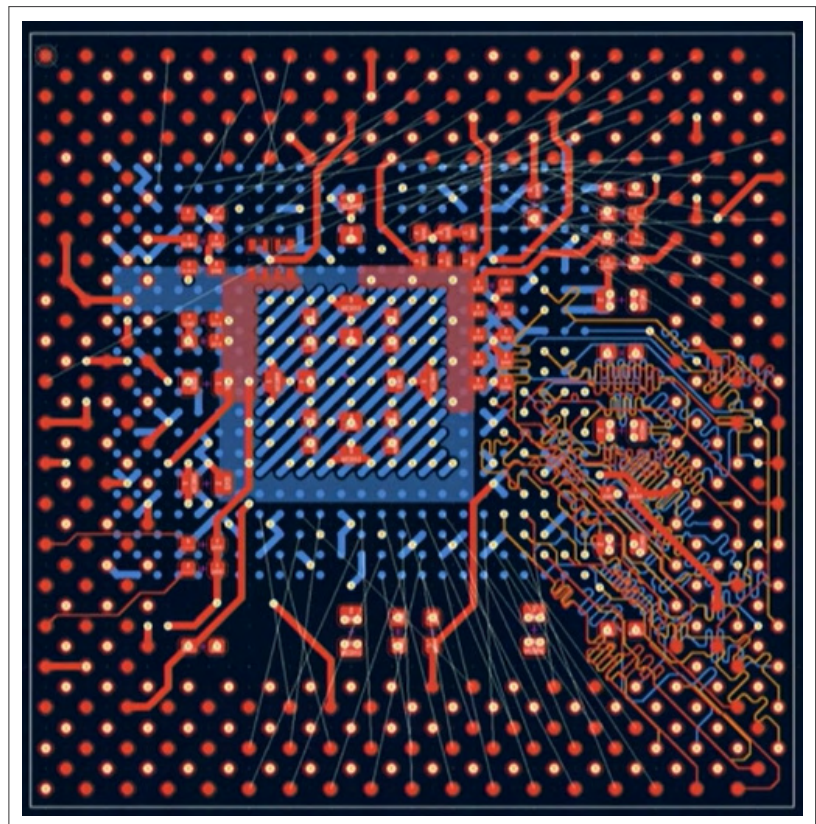
This talk details the author's journey into hardware modification, hacking and reverse-engineering of the Xbox console. Starting with only a background in software reverse engineering, the projects centred around the Xbox ramp-up, from adding a couple resistors to an open-source serial board all the way to building a custom CPU interposer and changing out the CPU that ships with the retail Xbox. The talk offers a small glimpse into the learning curve and experiments over ~4,000 hours of hobby exploration.

In addition to Xbox-specific projects, the speaker highlights learning opportunities and easy first hardware projects. Additionally, there are glimpses shared of how the newly-acquired hardware expertise can augment the traditional software reverse engineering process.

TAKEAWAYS:

- ✓ **In addition to entertainment value**, this talk highlights the benefits of adding cross-discipline skills. Being comfortable jumping between typical firmware reversing and hardware manipulation provides more paths forward beyond a roadblock.
- ✓ **The progression from 0 to 4,000 hours** of experimentation and self-study shows that dedication and drive is a key element of success and depth of understanding. Putting in the work can lead to impressive results.

Figure 7.
A KiCad routing diagram of the custom-designed and built CPU interposer for the Xbox.



Wallbleed: A Memory Disclosure Vulnerability in the Great Firewall of China

Authors: Shencha Fan, Jackson Sippe, Sakamoto San, Jade Sheffey, David Fifield, Amir Houmansadr, Elson Wedwards, and Eric Wustrow



This research explored a vulnerability in one of the DNS injectors that comprise the great firewall of China (GFW) that would send additional bytes from memory in specially-crafted DNS queries. For just over two years, this research collected over 5.1 billion responses and analysed the leaked data, gaining insights into how the GFW is designed, operates, and where it influences network connectivity.

The vulnerable injector, dubbed “Injector 3” sits on-path and responds to any DNS queries for a blocked domain before the legitimate DNS server – giving the client resolver a bogus IP. As the GFW injectors work in a bidirectional manner, clients inside of China connecting out will receive the same responses as an external client trying to resolve a hostname from a Chinese IP. By using Chinese IPs that were not serving as DNS servers, a US-based sensor would receive the injected DNS answers without interrupting normal traffic flows.

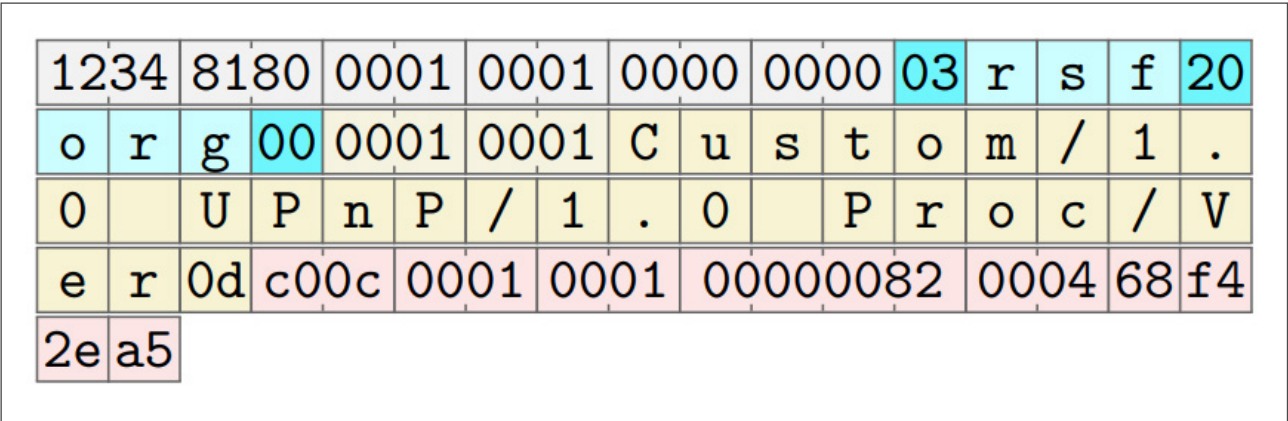
Partway through the monitoring, a partial patch for the memory leak was deployed, but still allowed for memory leaks. Watching in real time as the patches were deployed across the injectors highlighted the high-availability of the system. The bug was finally fully patched in March 2024.

By analysing the leaked data, it was clear that the injector is used for other traffic applications, as UPnP and HTTP patterns were found in the leaked data. The researchers were able to confirm that there was a short period of time where other DNS queries seen by the injector would have been leaked – generally less than 10 seconds. Finally, the researchers analysed the IP space that the DNS injector would respond to, finding IPs outside of China that were impacted – likely due to traffic being routed through China.

Figure 8.
A DNS answer for a blocked domain showing additional bytes of injector memory.

TAKEAWAYS:

- ✓ **This research** offered a fascinating glimpse into a part of the GFW, as well as how it can have impacts outside of the GFW’s intended geographic boundaries.
- ✓ **Keep in mind** that transient routing updates can send your organisation’s data through surprising jurisdictions, placing it subject to any filtering or monitoring.



Tblisi, Georgia. Image by
La'eeqah Galant (Thinkst)

Scaling software (in)security

- ✓ *Low-Effort Denial of Service with Recursion*
- ✓ *Is this memory safety here in the room with us?*
- ✓ *How to gain code execution on millions of people and hundreds of popular apps*
- ✓ *Node is a loader*
- ✓ *Mixing up Public and Private Keys in OpenID Connect deployments*

Low-Effort Denial of Service with Recursion

Authors: Alexis
Challande and Brad
Swain



[Paper](#)



[Video](#)

This research explored how recursion has been implemented in parsing routines of production systems, such as Elastic and Google's Protocol Buffers. These recursive implementations allow for single inputs to cause denial-of-service (DoS) attacks by deeply nesting inputs. Using CodeQL to search Java source of large and mature projects, the authors found numerous projects vulnerable to single-input DoS with multiple attack surfaces.

Despite limiting the CodeQL query to only singly-nested co-recursion (where function A calls function B which calls back to function A) and only Java implementations, the pattern was found to be surprisingly common. The authors advise developers to use an iterative approach for handling nested formats, or at least keep a depth counter that traps when it is found to be too large – before overflowing the stack frame.

TAKEAWAYS:

- ✓ **Recursion is often the simplest way** to conceptualise a parsing problem with nested structures. This research highlights that the simplest path is not always the best, and that recursion has no place in handling untrusted input.
- ✓ **Too often recursion is put on a pedestal of cleverness** in computer science education, which leads to its improper use in production. While it can be possible to implement recursion safely, the caveats are too many to allow through a code review unchecked.

Figure 9.
The final CodeQL
query to find recursive
function calls.



```
class RecursionSource extends MethodCall {
    RecursionSource() { not isTestPackage(this.getCaller().getDeclaringType()) }
}

module RecursiveConfig implements DataFlow::StateConfigSig {
    class FlowState = Method;

    predicate isSource(DataFlow::Node node, FlowState state) {
        node.asExpr() instanceof RecursionSource and
        state = node.asExpr().(MethodCall).getCaller()
    }

    predicate isSink(DataFlow::Node node, FlowState state) {
        node.asExpr() instanceof RecursionSource and
        state.calls+(node.asExpr().(MethodCall).getCaller()) and
        node.asExpr().(MethodCall).getCallee().calls(state)
    }
}

module RecursiveFlow = DataFlow::GlobalWithState<RecursiveConfig>;

import RecursiveFlow::PathGraph

from RecursiveFlow::PathNode source, RecursiveFlow::PathNode sink
where RecursiveFlow::flowPath(source, sink)
select sink.getNode(), source, sink, "Found a recursion: "
```

Is this memory safety here in the room with us?

Author: Thomas Dullien (Halvar Flake)



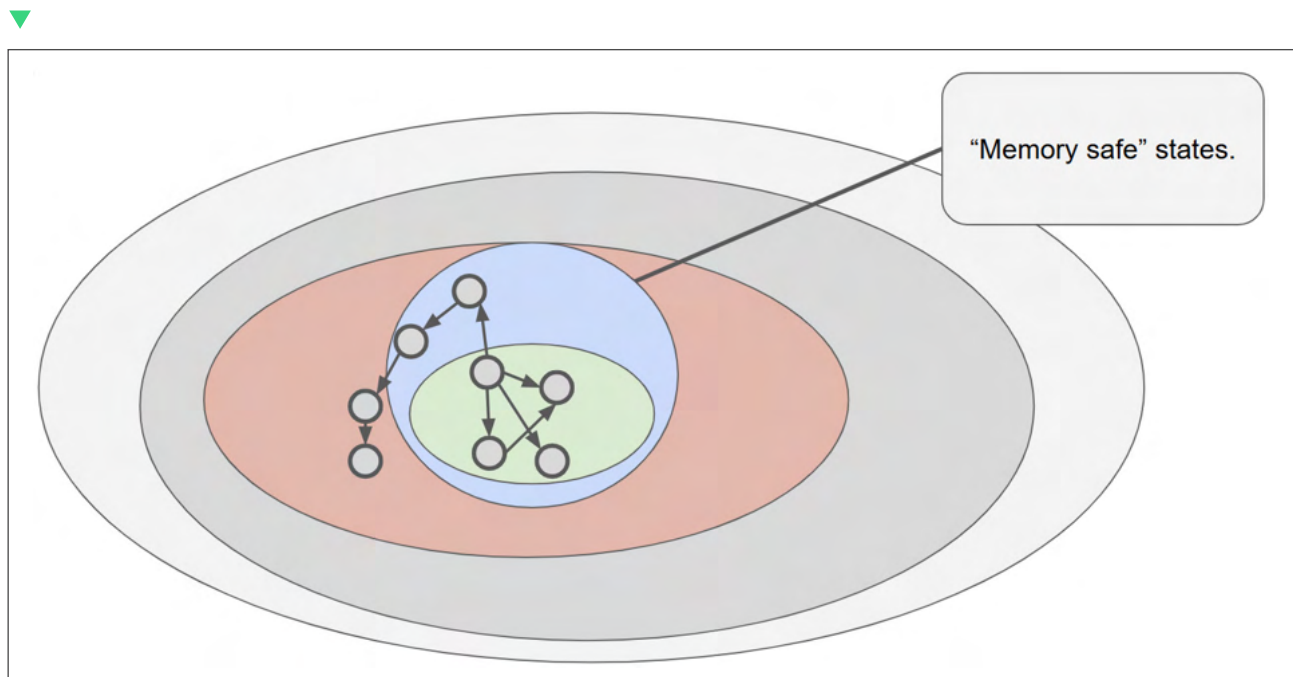
This keynote covered the lay of the land in 2025 vis-a-vis memory corruption. Starting with a succinct theoretical description of a vulnerability or exploit, the presentation quickly moved into looking at the pros and cons of five different approaches to solving memory safety, as well as looking at hardware/software co-design. Of these, the presenter was partial to compile-time, type-checker based approaches (e.g., in Rust). By using a more powerful type and ownership model, the scope of determining memory safety is reduced to a local issue, which can be recomposed to a global invariant. Despite this being a strong approach, the speaker notes that Rust is not a silver bullet, with weaknesses stemming from the development learning curve and the difficulty in writing safe `unsafe` code.

In 2025, the author states that userspace daemons should not have memory safety issues, assuming enough investment in securing them. Drivers, IPC using shared memory, and multi-language applications are still unsolved problems, but there are compelling research thrusts that may bring us to a place where memory corruption vulnerabilities are comparatively rare.

Figure 10.
A representation of possible CPU states: green being intended by the developer, blue states that respect memory safety, and red being attacker goal states.

TAKEAWAYS:

- ✓ **While this keynote** did not break new research, it drove home that, in 2025, the vast majority of applications should not have memory safety issues.
- ✓ **That memory corruption vulnerabilities** are still common is a strong indication that the vendor is not investing in modern development practices.



How to gain code execution on millions of people and hundreds of popular apps

Author: Eva  [Blog](#)

Node is a loader

Author: Tom Steele  [Blog](#)

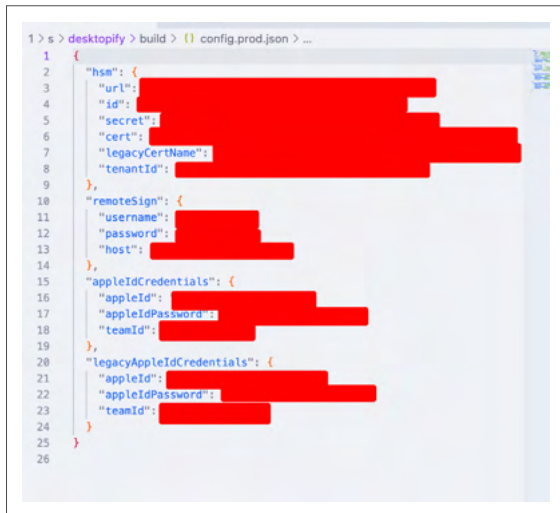


Figure 11.
A sanitised screenshot
of the credentials
hosted by the
ToDesktop container.

These blog posts look at how Electron applications influence endpoint security. **The first blog** detailed the process of finding a vulnerability in the ToDesktop platform. ToDesktop manages building web applications into native desktop applications via Electron. In addition to simplifying the build process and providing desktop-native UX compatibility, ToDesktop handles seamless updates of all ToDesktop applications. Some very popular applications rely on ToDesktop, likely running on millions of machines.

An initial exploration of the ToDesktop Firebase database did reveal an unused but writable table, but the author quickly pivoted to the container that builds ToDesktop applications. The author added a reverse shell to the post-install step of the build process for a test application they were building, and were able to explore the container. Sure enough, credentials for publishing all ToDesktop updates were readable. As a final test, the author used those credentials to deploy an update to their own test application, proving RCE.

The second blog shows how the popular Node.js application can be extended with native addons (loaded with the JS `require` keyword). Node is commonly used as part of modern applications, such as Slack or Adobe Photoshop, so it is often present and allows execution even in hardened environments. As a proof-of-concept, the author replaced an existing addon shipped with Slack with their DLL and gained execution on startup. Finally, the blog mentioned that this technique is already being used in the wild by malicious actors, and notes that there are some integrity features that could prevent the wholesale replacement of addons.

TAKEAWAYS:

- ✓ **Software monocultures** have been considered a threat for over two decades. What we're seeing now is better described as software hyper-polyculture: there are so many libraries and tools widely available that are consumed into software en masse that it doesn't take much to find a vulnerable component that can take down everything.
- ✓ **As software written in all languages** can increasingly be run everywhere, from in the browser and the backend to on the desktop, development becomes an effort in mosaic construction. If any of the components or processes involved are vulnerable, all the downstream software is at risk.
- ✓ **Node is incredibly prevalent** on modern endpoints, so much so that blocking it from loading would prevent the usage of many commercial applications. That these addons are as easy to drop and execute as they are highlight the permeability of application allow-listing. If your organisation relies on application allow-listing as part of the security perimeter, consider adding additional integrity checks on the scripts and addons that Node is permitted to execute.

Mixing up Public and Private Keys in OpenID Connect deployments

Author: Hanno Böck



[Blog](#)



[Code](#)

This blog looks at a number of rare cryptographic errors made in deployment of the OpenID federated login protocol. While examining a corpus of ~13,000 OpenID endpoint configurations, the author noted that a small number of them were misconfigured and the tooling failed to prevent such catastrophic errors. One of the worst errors (found in only 0.25% of deployments) was the private key shared publicly. As shown in the figure, the only difference for many common encryption algorithms when encoded into JSON for OpenID use is the presence or absence of a "d" field. As the public key is a subset of the data in the private key, the OpenID code will accept a private key without complaint – but with a private key, an attacker could sign tokens for any user that trusts that OpenID IdP.

Other problems noted included the use of known private keys, such as examples from documentation, or very weak encryption keys. The author notes that the specification for the JSON web key format was standardised many years after 512-bit RSA keys were considered unsafe, and they are still allowed (and used).

TAKEAWAYS:

- ✓ **Open standards and platforms** have the ability to be opinionated when it comes to security, and far too often they end up leaning towards allowing too much.
- ✓ **Any organisation** implementing a standard or using a platform should raise the bar to their own level of comfort, and not accept out-of-the-box defaults.

Figure 12.
An example of a public/private keypair as represented in the JSON web key format.



Here is an example of an ECDSA public key in JSON Web Key format:

```
{
  "kty": "EC",
  "crv": "P-256",
  "x": "MKBCTNIcKUSDii11ySs3526iDZ8AiTo7Tu6KPAqv7D4",
  "y": "4Et16SRW2YiLUrN5vfvVHuHp7x8Px1tmWWlbbM4IFyM"
}
```

Here is the corresponding private key:

```
{
  "kty": "EC",
  "crv": "P-256",
  "x": "MKBCTNIcKUSDii11ySs3526iDZ8AiTo7Tu6KPAqv7D4",
  "y": "4Et16SRW2YiLUrN5vfvVHuHp7x8Px1tmWWlbbM4IFyM",
  "d": "870MB6gfuTJ4HtUnUvYMyJpr5eUZNP4Bk43bVdj3eAE"
}
```



*Fynbos after the fires on
Devil's Peak, Cape Town. Image
by Roberto Aldera (Thinkst)*

Nifty sundries

- ✓ *Will It Run? Fooling EDRs With Command Lines Using Empirical Data*
- ✓ *Homoglyph-Based Attacks: Circumventing LLM Detectors*
- ✓ *28 Months Later - The Ongoing Evolution of Russia's Cyber Operations*
- ✓ *'It's Not Paranoia If They're Really After You': When Announcing Deception Technology Can Change Attacker Decisions*
- ✓ *Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack*

Will It Run? Fooling EDRs With Command Lines Using Empirical Data

Author: Wietze Beukema

 Website  Code  Video

Homoglyph-Based Attacks: Circumventing LLM Detectors

Author: Aldan Creo

 Paper  Code  Video

These talks explore how unicode characters can bypass detections. The first talk explores how common living-off-the-land command line tools handle unicode replacements or inserted characters. Many EDR detection rules perform regular expression matches on the command line strings, so finding ways to express the command and arguments in an obfuscated way can bypass those detections. For example, Microsoft's Defender tool blocks the use of certutil to download arbitrary data. By inserting unicode replacements (e.g., an em dash vs a hyphen), or adding extra double quotes in the middle of strings, Defender can be bypassed. Finally, the talk highlights common patterns of obfuscation and how to write rules to detect these.

The second talk looks at another unicode-based bypass: LLM text detection. This attack changes the tokenisation output of an LLM to replace characters with their homoglyphs (i.e., characters that look the same but are differently encoded). As output is typically using the most common character encoding for the language, detectors find the output with homoglyphs sufficiently different, despite looking identical to humans (e.g., a vs a – one is a latin 'a', the other a cyrillic 'a'). Perplexity-based measurements (how surprising a text is given a certain probability distribution) are confused as the homoglyphs appear rarely if never outside of this class of attack. An evaluation of many popular detectors found that all of them are tricked by such a simple substitution attack.

TAKEAWAYS:

- ✓ **The command-line evasion research** benefits both the offensive and defensive communities, though it is up to those communities to learn and adopt this research.
- ✓ **Relying on simple pattern-matching** of strings is bound to become a game of whack-a-mole, whereas looking for the events that those strings cause is a much more sane path.
- ✓ **We've seen similar unicode confusions** hitting phishing URLs and source code (e.g., where a malicious patch looks benign when viewed) in the past. This class of confusion where one entity understands content differently than another has legs. Keep an eye out for more domains where these tricks bear fruit.

Figure 13.
A table showing some commonly-abused built-in Windows utilities and the classes of command-line obfuscation they allow.

Executable	Obfuscation types	Character Substitution	Character Insertion	Option Character Substitution	Quote Insertion	Shorthands	File Path Transformations	URL Transformations	RaNdOmCaSe
certreq.exe	7	✓	✓	✓	✓		✓	✓	✓
reg.exe	6	✓	✓	✓	✓		✓		✓
expand.exe	6	✓	✓	✓	✓		✓		✓
certutil.exe	6	✓	✓	✓	✓		✓		✓
runas.exe	5		✓		✓	✓	✓		✓
robocopy.exe	5	✓		✓	✓		✓		✓
regsvr32.exe	5		✓	✓	✓		✓		✓
regedit.exe	5		✓	✓	✓		✓		✓
powershell.exe (and push)	5			✓	✓	✓	✓		✓
ping.exe	5		✓	✓	✓			✓	✓
nslookup.exe	5		✓	✓	✓	✓			✓
netstat.exe	5	✓	✓	✓	✓				✓
msiexec.exe	5	✓		✓	✓		✓		✓
msbuild.exe	5			✓	✓		✓	✓	✓
makecab.exe	5	✓		✓	✓		✓		✓
ipconfig.exe	5	✓	✓	✓	✓				✓
findstr.exe	5	✓		✓	✓		✓		✓
extract32.exe	5	✓		✓	✓		✓		✓
dism.exe	5		✓	✓	✓		✓		✓

28 Months Later - The Ongoing Evolution of Russia's Cyber Operations

Author: The Grugq



[Slides](#)



[Video](#)

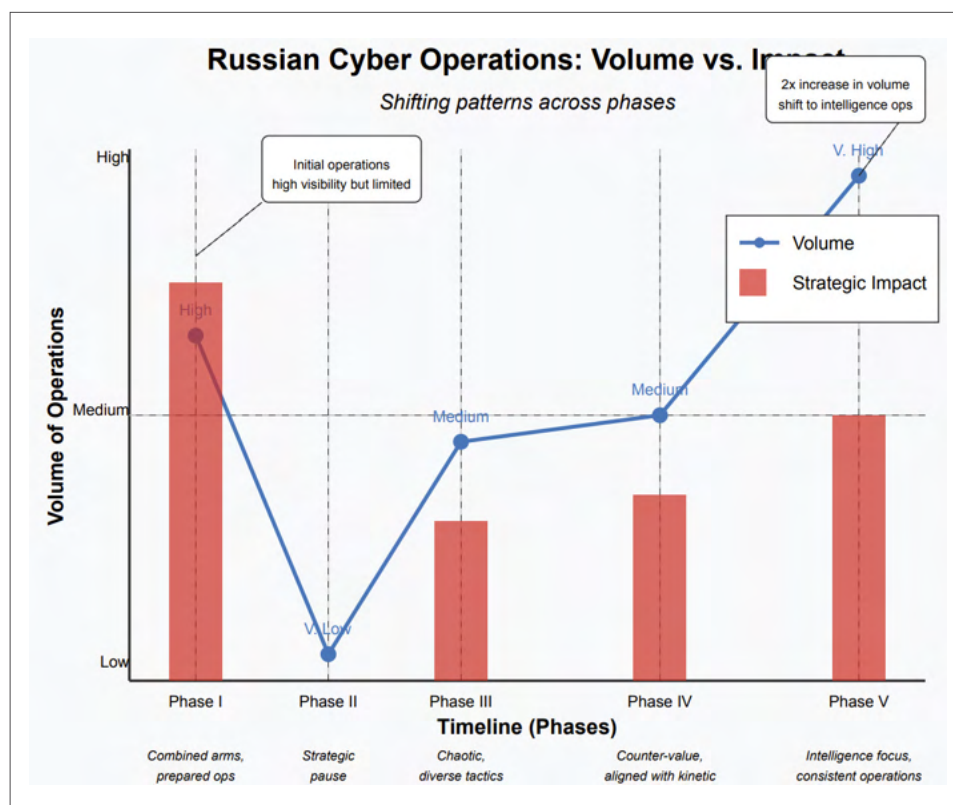
This talk examined the shifting strategies and tactics of Russia's cyber attacks following their invasion of Ukraine. While the early days of the kinetic invasion were coupled with cyber effects, very quickly the pre-planned "cyber-munitions" and accesses were depleted. Most likely because Russia assumed they would be more successful than they have been, there was a period of ineffective operations presumed to be done purely for performative reasons.

Over time, more tightly integrated operations with the military had more effect (targeting critical infrastructure). The author notes, however, that in the later stages of the invasion, the majority of cyber operations returned to espionage. Effect-based operations usually destroy access, so burning access for limited return has shown to be unpopular – covert access for intelligence gain over time has been a priority in the later stages of the conflict.

TAKEAWAYS:

- ✓ **While most organisations** will not be directly involved in an escalated cyber and kinetic conflict, there are valuable insights to draw from this analysis. Both the aggressor in this conflict and those who are learning from them will note that cyber eventually returns to an espionage vector.
- ✓ **Unless your organisation** is part of a nation's critical infrastructure, expect longer-term, covert breaches as opposed to destructive effects. The upshot of this is that the defenses against non nation-state actors overlap with those of espionage defense. Protecting sensitive data, improving detections and evicting adversaries from networks will pay dividends against threats, ranging from ransomware gangs to nation-state actors.

Figure 14.
A qualitative
chart showing the
progression of Russian
cyber operations in
Ukraine.



‘It’s Not Paranoia If They’re Really After You’: When Announcing Deception Technology Can Change Attacker Decisions

Authors: Andrew Reeves and Debi Ashenden



Paper

This research explored how the awareness of deployed deception (such as honeypots) influenced an attacker’s behaviour. Using red teamers and other professional offensive security participants, a representative network was scanned and loaded into the Blood-Hound graph visualisation tool. These graphs were distributed ahead of the experiment to the participants who were asked to develop an attack path to attain domain administrator permissions.

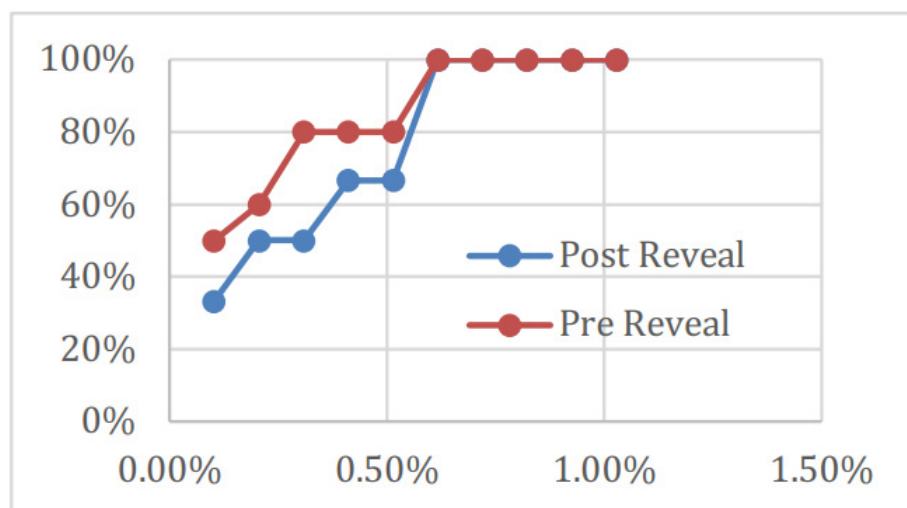
During a conversational survey with each participant, the rationales for their chosen attack paths were collected. The participant was then informed that decoys had been deployed in the network, and asked how that would (or would not) impact their planned attack path. 40% of participants were unable to create a new attack path. Of those who did, the secondary paths were longer, but did evade detection in low-density (<0.6% decoy) networks – a slight improvement versus when not prepared to encounter deception.

The paper is upfront about a number of experimental shortcomings, from the small sample size, to questions of how close the behaviour of red teamers is to that of real attackers, and assuming that attackers would be undetected as they transit through real endpoints. Despite those shortcomings, this work offers more clarity about how offensive operators operate under differing assumptions about the state of the network under attack.

TAKEAWAYS:

- ✓ **This research shows** that pure messaging deception (e.g., posting a purchase of a deception technology) can influence attackers who perform preliminary OSINT. An attacker who is primed to consider the risks of detection from decoys will be more likely to avoid real misconfigurations or security weaknesses that are obvious.
- ✓ **This research indicates that**, if deception is being employed, slightly higher density is needed if attackers are aware of its presence. However, the study ignored the possibility of detection while laterally moving across the network on real assets.
- ✓ **Driving an attacker** to take longer paths can increase the chance of detection – a win.

Figure 15. A chart showing how attack paths intersected with decoy/honeypots as the density of decoys increased for both attackers aware of and ignorant of decoy deployment.



Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack

Authors: Ziqiang Wang, Xuwei Feng, Qi Li, Kun Sun, Yuxiang Yang, Mengyuan Li, Ganqiu Du, Ke Xu, and Jianping Wu



[Paper](#)



[Code](#)

This research explored leveraging side-channels to hijack TCP sessions from a victim on the same (encrypted) Wi-Fi network. It identified different packet responses from a server based on the TCP sequence number and acknowledgement received. Based on the response type, the response packet will be a unique, fixed size.

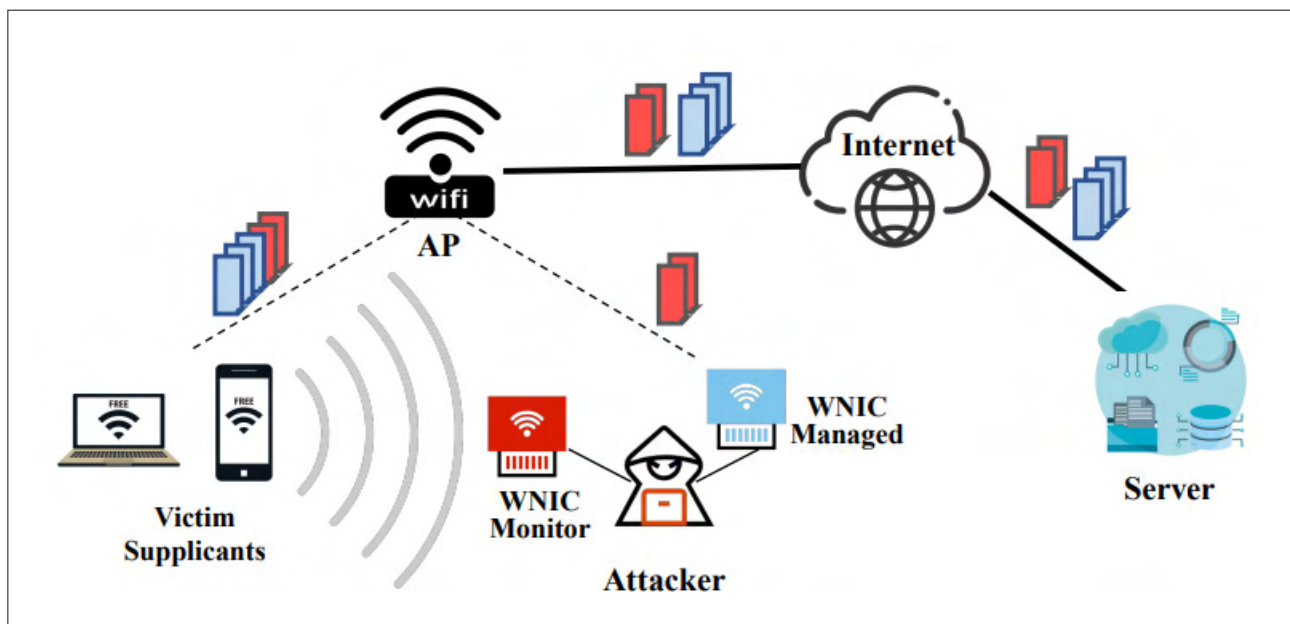
An attacker that is both joined to the network (e.g., in a co-working space or coffee shop) and listening to the raw Wi-Fi frames, then can determine the response type. These different response types indicate if the attacker-guessed values are too high, too low or within the acceptable window range.

By successfully determining the values, the attacker can send RST packets to the server – severing the connection. The researchers demonstrated two attacks, a DoS against SSH (by simply sending an accepted RST to the server) and injecting content into a long-lived HTTP connection. These attacks took less than 30 seconds to complete, and worked on 60 to 80% of tested network configurations.

TAKEAWAYS:

- ✓ **The highest-impact attack** (HTTP injection) only works for unsecured connections. It's a good reminder in a remote/hybrid work world that VPNs are a good layer of protection on untrusted networks.
- ✓ **This research highlights the tension** between features for performance and flow-control, and the security implications.
- ✓ **The immense configurability of modern systems** means that, while there is likely a better fit for a specific environment, determining that is nigh impossible, and almost certainly not right-sized.

Figure 16.
A figure showing the high-level attack scenario and threat model.





Natal Midlands, South Africa.
Image by mh (Thinkst)

Conclusions

That's all for this quarter.

WE HIGHLIGHTED THREE THEMES FOR THIS QUARTER:

1. Putting theory into practice.
2. Deeply understanding systems, even down to the hardware level.
3. Scaling up software (in)security.

We're looking forward to seeing what comes next in 2025. We'll be back next time with more picks from great researchers.

